

Learn Python for Automation

Lesson 5: File I/O and CSV Automation

File I/O and CSV Automation - Study Notes

Key Concepts

1. **Text File Operations** – opening, reading, writing, and closing files safely with context managers (`with` statement).
2. **CSV Parsing with the `csv` Module** – using `csv.reader`, `csv.writer`, and `csv.DictReader`/`csv.DictWriter` for robust, delimiter aware I/O.
3. **Data Cleanup Basics** – trimming whitespace, handling missing values, and converting data types on the fly.
4. **Error Handling & File Modes** – understanding `read` (`'r'`), `write` (`'w'`), `append` (`'a'`), and binary modes, plus handling `FileNotFoundError` and `UnicodeDecodeError`.
5. **File Paths and Cross Platform Compatibility** – using `os.path` or `pathlib` to build portable paths.

Summary

File I/O is the foundation of any data driven Python program. By mastering the `with open(...)` pattern, you ensure files are closed automatically and exceptions are handled gracefully. The `csv` module abstracts away the intricacies of comma separated data, providing a clean API for reading rows as lists or dictionaries and writing rows back to disk.

Data cleanup is often the first step in any data pipeline: trimming whitespace, standardizing missing value markers, and converting strings to numeric types. Even simple scripts that read a CSV, clean each row, and write a new CSV can save hours of manual work. These skills are essential for beginners because they expose students to real world data handling tasks while reinforcing core Python concepts such as loops, conditionals, and exception handling.

Important Points

- **Always use `with open(...)` as f:** – guarantees file closure and simplifies error handling.
- **Specify the correct mode:**
 - `'r'` – read (default)
 - `'w'` – write (truncates file)
 - `'a'` – append
 - `'b'` – binary (e.g., `'rb'`, `'wb'`)
- **`csv.reader` vs `csv.DictReader`:**
 - `reader` returns a list of strings per row.
 - `DictReader` uses the first row as header keys, returning a dict per row.
- **Delimiter and quoting:** `csv.writer(..., delimiter=',', quotechar='"', quoting=csv.QUOTE_MINIMAL)` ensures proper handling of embedded commas or newlines.
- **Data cleanup:**

- ``row = [cell.strip() for cell in row]`` – removes leading/trailing whitespace.
- Convert numeric strings: ``int(cell)`` or ``float(cell)`` with ``try/except`` for robustness.
- Replace missing values: ``cell`` or ``N/A``.
 - **Cross platform paths**: use ``pathlib.Path`` for readability (``Path('data') / 'file.csv'``).
 - **Always close files** (automatically via ``with``).
 - **Handle exceptions**: ``FileNotFoundError``, ``UnicodeDecodeError``, ``csv.Error``.

Examples

1. Reading a CSV and Printing Cleaned Rows

```
```python
import csv
from pathlib import Path
csv_file = Path('data') / 'raw.csv'
with csv_file.open(newline="", encoding='utf-8') as f:
 reader = csv.DictReader(f)
 for row in reader:
 # Basic cleanup
 cleaned = {k: v.strip() for k, v in row.items()}
 print(cleaned)
```
```

***Explanation*:**

- ``newline=""`` prevents Python from altering newline characters.
- ``DictReader`` automatically maps headers to values.
- ``strip()`` removes surrounding whitespace.

2. Writing a Cleaned CSV

```
```python
import csv
from pathlib import Path
input_file = Path('data') / 'raw.csv'
output_file = Path('data') / 'cleaned.csv'
with input_file.open(newline="", encoding='utf-8') as fin, \
 output_file.open('w', newline="", encoding='utf-8') as fout:
 reader = csv.DictReader(fin)
 fieldnames = reader.fieldnames
 writer = csv.DictWriter(fout, fieldnames=fieldnames,
 quoting=csv.QUOTE_MINIMAL)
 writer.writeheader()
 for row in reader:
 cleaned = {k: v.strip() for k, v in row.items()}
 # Convert numeric fields if needed
```

```

if 'age' in cleaned:
 try:
 cleaned['age'] = int(cleaned['age'])
 except ValueError:
 cleaned['age'] = None
writer.writerow(cleaned)
...

```

**\*Explanation\*:**

- The writer uses the same headers as the reader.
- Numeric conversion is attempted; on failure, `None` is written (which becomes an empty cell).
- The `with` block ensures both files close automatically.

---

## Quick Review

1. What is the purpose of the `with` statement when working with files?
2. How does `csv.DictReader` differ from `csv.reader`?
3. Why should you use `newline=""` when opening a CSV file for reading or writing?
4. Give an example of a simple data cleanup operation you might perform on a CSV row.
5. Which Python module provides functions for handling file paths in a platform independent way?

---

## Further Reading

- **\*\*Python `csv` module documentation\*\*** – advanced options like `dialect`, `escapechar`, and custom delimiters.
- **\*\*`pathlib` vs `os.path`\*\*** – modern path handling.
- **\*\*Error handling in file I/O\*\*** – using `try/except` blocks and custom error messages.
- **\*\*Pandas basics\*\*** – for more powerful CSV manipulation once you're comfortable with the basics.
- **\*\*Unicode and encoding\*\*** – understanding UTF 8, UTF 16, and how to handle non ASCII data.

---

